

AI Applications Lecture 8

Evaluation of Probabilistic Language Models

SUZUKI, Atsushi^a

^aI thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
5. How do we evaluate a probabilistic language model?
6. How do we evaluate natural language outputs?
7. Summary
8. Next Time

Review of the previous lecture

In the previous lectures, we learned about the **natural language sequence generation pipeline**, including neural networks and tokenization.

Review of the previous lecture

In the previous lectures, we learned about the **natural language sequence generation pipeline**, including neural networks and tokenization.

We also provided a rigorous formulation of:

- **probabilistic language models**
- **token generators** based on **sampling**, **greedy search**, and **beam search**.

Review of the previous lecture

In the previous lectures, we learned about the **natural language sequence generation pipeline**, including neural networks and tokenization.

We also provided a rigorous formulation of:

- **probabilistic language models**
- **token generators** based on **sampling**, **greedy search**, and **beam search**.

In this lecture, we will focus on **evaluation**, addressing how to **automatically and quantitatively** evaluate both probabilistic language models and natural language inputs/outputs.

Learning outcomes

Through this lecture, students should be able to:

- Explain the **non-triviality** of evaluation in natural language processing compared to evaluation in classical supervised machine learning.

Learning outcomes

Through this lecture, students should be able to:

- Explain the **non-triviality** of evaluation in natural language processing compared to evaluation in classical supervised machine learning.
- Distinguish between the **evaluation of natural language string input/output** and the **evaluation of probabilistic language models**.

Learning outcomes

Through this lecture, students should be able to:

- Explain the **non-triviality** of evaluation in natural language processing compared to evaluation in classical supervised machine learning.
- Distinguish between the **evaluation of natural language string input/output** and the **evaluation of probabilistic language models**.
- Evaluate natural language string input/output using probabilistic language models with **appropriate metrics**.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. The Importance of Automatic Evaluation

4. What do we evaluate?

5. How do we evaluate a probabilistic language model?

6. How do we evaluate natural language outputs?

7. Summary

8. Next Time

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. The Importance of Automatic Evaluation

4. What do we evaluate?

5. How do we evaluate a probabilistic language model?

6. How do we evaluate natural language outputs?

7. Summary

8. Next Time

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} := \{1, \dots, k\}$

Function:

- $f : \mathcal{X} \rightarrow \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x .

Definition:

- (LHS) $:=$ (RHS): Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\mathbf{a} = (a_1, a_2, \dots)$.

- A function $\mathbf{a} : [1, n]_{\mathbb{Z}} \rightarrow \mathcal{A}$.
- Length is denoted by $|\mathbf{a}|$.

Vector: Denoted by $\mathbf{v}$.

- A column of numbers, $\mathbf{v} \in \mathbb{R}^n$.
- i -th element is v_i .

Matrix: Denoted by $\mathbf{A}$.

- $m \times n$ matrix: $\mathbf{A} \in \mathbb{R}^{m,n}$.
- (i, j) -th element is $a_{i,j}$.
- Transpose: $\mathbf{A}^{\top}$.

Tensor: Denoted by $\underline{\mathbf{A}}$.

- Simply a multi-dimensional array.
- Vector $\rightarrow$ 1st-order, Matrix $\rightarrow$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. The Importance of Automatic Evaluation

4. What do we evaluate?

5. How do we evaluate a probabilistic language model?

6. How do we evaluate natural language outputs?

7. Summary

8. Next Time

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. The Importance of Automatic Evaluation

4. What do we evaluate?

5. How do we evaluate a probabilistic language model?

6. How do we evaluate natural language outputs?

7. Summary

8. Next Time

Question: What is the difficulty in evaluating LLMs?

The importance of automatic evaluation

When evaluating computational methods, it is practically useful to employ **automatic and quantitative** evaluation whenever possible.

The importance of automatic evaluation

When evaluating computational methods, it is practically useful to employ **automatic and quantitative** evaluation whenever possible.

Automatic evaluation does not require human resources and also contributes to ensuring the **reproducibility** of experiments.

Formulation of automatic evaluation in classical supervised learning

How can we evaluate the AI automatically?

Formulation of automatic evaluation in classical supervised learning

How can we evaluate the AI automatically?

If a correct input-output dataset and an output evaluation function are provided, the simple **average evaluation** works.

Formulation of automatic evaluation in classical supervised learning

How can we evaluate the AI automatically?

If a correct input-output dataset and an output evaluation function are provided, the simple **average evaluation** works.

Definition (Framework for Classical Evaluation)

Given an input space $\mathcal{X}$, an output space $\mathcal{Y}$, a trained map $f : \mathcal{X} \rightarrow \mathcal{Y}$, and an evaluation function $E : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$. For a test dataset $\{(x_i, y_i)\}_{i=1}^N$, the evaluation value is defined as

$$\text{Eval}(f) := \frac{1}{N} \sum_{i=1}^N E(f(x_i), y_i). \quad (1)$$

Here, $N \in \mathbb{Z}_{>0}$ is the number of test points, and E can be a **loss** (smaller is better) or a **score** (larger is better).

Formulation of automatic evaluation in classical supervised learning

Remark

Typical examples include the **squared Euclidean distance** $E(\hat{\mathbf{y}}, \mathbf{y}) = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$ when $\mathcal{Y} = \mathbb{R}^d$, and the **0-1 loss** $E(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$ when $\mathcal{Y}$ is a finite set.

Example (Calculation Example of Squared Error and 0-1 Loss)

Regression: If $\hat{\mathbf{y}} = (2, 0)^\top$, $\mathbf{y} = (1, 1)^\top$, then

$$\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = (2 - 1)^2 + (0 - 1)^2 = 1 + 1 = 2. \quad (2)$$

Classification: If $\hat{y} = \text{cat}$, $y = \text{dog}$, then $E(\hat{y}, y) = 1$.

Formulation of automatic evaluation in classical supervised learning

Exercise (Exercise on Classical Evaluation)

(1) Find the squared distance between $\hat{\mathbf{y}} = (3, -1)^\top$ and $\mathbf{y} = (1, 2)^\top$.

Formulation of automatic evaluation in classical supervised learning

Exercise (Exercise on Classical Evaluation)

(1) Find the squared distance between $\hat{\mathbf{y}} = (3, -1)^\top$ and $\mathbf{y} = (1, 2)^\top$. (2) Find the 0-1 distance for $\hat{y} = A, y = B$.

Formulation of automatic evaluation in classical supervised learning

Answer

(1) Step-by-step calculation of squared distance:

- First, find the difference vector: $\hat{\mathbf{y}} - \mathbf{y} = (3 - 1, -1 - 2)^\top = (2, -3)^\top$.

Formulation of automatic evaluation in classical supervised learning

Answer

(1) Step-by-step calculation of squared distance:

- First, find the difference vector: $\hat{\mathbf{y}} - \mathbf{y} = (3 - 1, -1 - 2)^\top = (2, -3)^\top$.
- Then, calculate the squared norm:

$$\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = (2)^2 + (-3)^2 = 4 + 9 = 13.$$

Formulation of automatic evaluation in classical supervised learning

Answer

(1) Step-by-step calculation of squared distance:

- First, find the difference vector: $\hat{\mathbf{y}} - \mathbf{y} = (3 - 1, -1 - 2)^\top = (2, -3)^\top$.
- Then, calculate the squared norm:

$$\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = (2)^2 + (-3)^2 = 4 + 9 = 13.$$

(2) Step-by-step calculation of 0-1 distance:

- Since the prediction $\hat{y}$ and the true label y do not match, the indicator function is true: $\mathbf{1}[\hat{y} \neq y] = 1$.

Why automatic evaluation of natural language output is difficult

In the space of natural language strings, **semantic equivalence** is essential, and there can be **infinitely many correct answers**.

Why automatic evaluation of natural language output is difficult

In the space of natural language strings, **semantic equivalence** is essential, and there can be **infinitely many correct answers**.

Therefore,

- We CANNOT prepare all possible correct answers on the test side.

Why automatic evaluation of natural language output is difficult

In the space of natural language strings, **semantic equivalence** is essential, and there can be **infinitely many correct answers**.

Therefore,

- We CANNOT prepare all possible correct answers on the test side.
- We CANNOT automatically determine the semantic equivalence between input strings.

Why automatic evaluation of natural language output is difficult

In the space of natural language strings, **semantic equivalence** is essential, and there can be **infinitely many correct answers**.

Therefore,

- We CANNOT prepare all possible correct answers on the test side.
- We CANNOT automatically determine the semantic equivalence between input strings.

For this reason, various **evaluation metrics** have been proposed.

Non-triviality of LLM evaluations - answer

Question: What is the difficulty in evaluating LLMs?

Answer:

Non-triviality of LLM evaluations - answer

Question: What is the difficulty in evaluating LLMs?

Answer: While automatic evaluations are preferred with the LLM's large scale considered, automatically evaluating natural language outputs is non-trivial. This requires us to consider various non-trivial evaluation metrics.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. The Importance of Automatic Evaluation

4. What do we evaluate?

5. How do we evaluate a probabilistic language model?

6. How do we evaluate natural language outputs?

7. Summary

8. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
- 4. What do we evaluate?**
5. How do we evaluate a probabilistic language model?
6. How do we evaluate natural language outputs?
7. Summary
8. Next Time

Question: When we evaluate an LLM, what do we evaluate?

Two major families: language model evaluation vs. string output evaluation

Evaluation methods can be broadly divided into the following two categories:

Two major families: language model evaluation vs. string output evaluation

Evaluation methods can be broadly divided into the following two categories:

Two major families: language model evaluation vs. string output evaluation

Evaluation methods can be broadly divided into the following two categories:

- **Evaluation of Probabilistic Language Models:**

Two major families: language model evaluation vs. string output evaluation

Evaluation methods can be broadly divided into the following two categories:

- **Evaluation of Probabilistic Language Models:**

- Examples include **perplexity** and the **most likely option** in multiple-choice tasks (MMLU, HellaSwag, etc.).
- **Pros:** Avoids the difficulties of string generation.
- **Cons:** Cannot measure the string output performance itself.

Two major families: language model evaluation vs. string output evaluation

Evaluation methods can be broadly divided into the following two categories:

- **Evaluation of Probabilistic Language Models:**
 - Examples include **perplexity** and the **most likely option** in multiple-choice tasks (MMLU, HellaSwag, etc.).
 - **Pros:** Avoids the difficulties of string generation.
 - **Cons:** Cannot measure the string output performance itself.
- **Evaluation of Natural Language String Input/Output:**

Two major families: language model evaluation vs. string output evaluation

Evaluation methods can be broadly divided into the following two categories:

- **Evaluation of Probabilistic Language Models:**

- Examples include **perplexity** and the **most likely option** in multiple-choice tasks (MMLU, HellaSwag, etc.).
- **Pros:** Avoids the difficulties of string generation.
- **Cons:** Cannot measure the string output performance itself.

- **Evaluation of Natural Language String Input/Output:**

- Examples include **Exact Match/F1**, **BLEU**, **ROUGE-L**, **BERTScore**, and **Accuracy** for math QA.
- **Pros:** Measures the output performance directly.
- **Cons:** Calculation method differs for each task and may have language dependencies.

Question: When we evaluate an LLM, what do we evaluate?

Answer:

Question: When we evaluate an LLM, what do we evaluate?

Answer: We may evaluate

- the probabilistic language model represented by LLM, or
- the outputs by LLM.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
- 4. What do we evaluate?**
5. How do we evaluate a probabilistic language model?
6. How do we evaluate natural language outputs?
7. Summary
8. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
- 5. How do we evaluate a probabilistic language model?**
6. How do we evaluate natural language outputs?
7. Summary
8. Next Time

Question: How do we convert the probability mass values into a score?

Perplexity

Dataset format: Multiple evaluation sentences are given. Each sentence is a tokenized sequence $t = (t_1, \dots, t_n)$.

Example (Sample from WikiText-2 [6] (English text))

Sample text:

Rifenburg lived 37 of his years in Buffalo . His wife , the former Jane Morris , was the head of the Buffalo Jills cheerleaders when they met . Rifenburg , who was survived by three sons , (Douglas

Perplexity

Idea: If a language model behaves well like a real language, it should **allocate a high probability to real sentences.**

Perplexity

Idea: If a language model behaves well like a real language, it should **allocate a high probability to real sentences**.

Therefore, we introduce **perplexity**, which evaluates how much **likelihood** the model assigns to the observed sequence, using a length-normalized average negative log-likelihood.

Perplexity

Definition (Rigorous Definition of Cross-Entropy and Perplexity)

Let the **vocabulary** set be $\mathcal{V}$, and a **token sequence** be $\mathbf{t} = (t_1, \dots, t_n)$. The **language model** P provides a probability distribution $P(\cdot \mid \mathbf{t}_{<i})$ over $\mathcal{V}$ for the next token, conditioned on previous tokens $\mathbf{t}_{<i} := (t_1, \dots, t_{i-1})$.

The **average negative log-likelihood** (token-level cross-entropy) is defined as

$$H(\mathbf{t}; P) := -\frac{1}{n} \sum_{i=1}^n \log P(t_i \mid \mathbf{t}_{<i}), \quad (3)$$

and the **perplexity** is defined as

$$\text{PPL}(\mathbf{t}; P) := \exp(H(\mathbf{t}; P)). \quad (4)$$

Perplexity

Remark

Note that differences in tokenization can lead to different results. This is also true for several other evaluation metrics.

Perplexity

Example (PPL Calculation)

Formal Setting:

- Vocabulary $\mathcal{V} = \{a, b\}$
- Sequence $t = (t_1, t_2, t_3) = (a, b, a)$
- Probabilities are given as:

$$P(a \mid ()) = 0.8$$

$$P(b \mid (a)) = 0.5$$

$$P(a \mid (a, b)) = 0.25$$

Here $n = 3$. Let's calculate the perplexity.

Perplexity

Calculation Steps:

First, we write down the formula for the average negative log-likelihood, $H(\mathbf{t}; P)$:

$$H(\mathbf{t}; P) = -\frac{1}{3} \left(\log P(t_1=a \mid ()) + \log P(t_2=b \mid (a)) + \log P(t_3=a \mid (a, b)) \right)$$

Perplexity

Calculation Steps:

Next, we plug in the given probability values:

$$\begin{aligned} H(t; P) &= -\frac{1}{3} \left(\log P(t_1=a \mid ()) + \log P(t_2=b \mid (a)) + \log P(t_3=a \mid (a, b)) \right) \\ &= -\frac{1}{3} (\log 0.8 + \log 0.5 + \log 0.25) \end{aligned}$$

Perplexity

Calculation Steps:

Using the property that the sum of logs is the log of the product:

$$\begin{aligned} H(\mathbf{t}; P) &= -\frac{1}{3} (\log 0.8 + \log 0.5 + \log 0.25) \\ &= -\frac{1}{3} \log(0.8 \times 0.5 \times 0.25) = -\frac{1}{3} \log 0.1. \end{aligned}$$

This is the value for $H(\mathbf{t}; P)$.

Perplexity

Calculation Steps:

Finally, we compute the perplexity by taking the exponential of H :

$$\text{PPL}(\mathbf{t}; P) = \exp\left(-\frac{1}{3} \log 0.1\right) = 0.1^{-1/3} \approx 2.154. \quad (5)$$

The perplexity is approximately 2.154.

Perplexity

Exercise (PPL Exercise)

For $t = (a, a)$, with $P(a \mid ()) = 0.6$ and $P(a \mid (a)) = 0.3$, calculate the PPL step-by-step.

Perplexity

Answer

Step-by-step Calculation: The length of the sequence is $n = 2$.

$$\begin{aligned} H(t; P) &= -\frac{1}{2} (\log P(t_1=a \mid ()) + \log P(t_2=a \mid (a))) \\ &= -\frac{1}{2} (\log 0.6 + \log 0.3) = -\frac{1}{2} \log(0.18). \end{aligned}$$

Therefore,

$$\text{PPL}(t; P) = \exp(H) = (0.18)^{-1/2} \approx 2.357.$$

Accuracy of the most likely option in a MCQ

The perplexity just measures the similarity between the language model and the real language.

It does NOT

Accuracy of the most likely option in a MCQ

The perplexity just measures the similarity between the language model and the real language.

It does NOT measure the input-output quality.

Accuracy of the most likely option in a MCQ

The perplexity just measures the similarity between the language model and the real language.

It does NOT measure the input-output quality.

When we want to measure the input-output quality, the problem is that

Accuracy of the most likely option in a MCQ

The perplexity just measures the similarity between the language model and the real language.

It does NOT measure the input-output quality.

When we want to measure the input-output quality, the problem is that scoring free-form generation is difficult.

Hence, we can measure it on

Accuracy of the most likely option in a MCQ

The perplexity just measures the similarity between the language model and the real language.

It does NOT measure the input-output quality.

When we want to measure the input-output quality, the problem is that scoring free-form generation is difficult.

Hence, we can measure it on **multiple choice questions (MCQs)**.

Example of multiple-choice datasets

Example (ARC-Easy [1] Data Example)

Science question (multiple-choice)

Input (question): Which is the function of the gallbladder?

Input (choices): A: store bile, B: produce bile, C: store digestive enzymes, D: produce digestive enzymes

Example of multiple-choice datasets

Example (ARC-Easy [1] Data Example)

Science question (multiple-choice)

Input (question): Which is the function of the gallbladder?

Input (choices): A: store bile, B: produce bile, C: store digestive enzymes, D: produce digestive enzymes

Output (gold label): A

Example of multiple-choice datasets

Example (ARC-Challenge Data Example)

More difficult science question (multiple-choice)

Input (question): Cities control the amount of pollution that is allowed to come from cars. How does this most likely help people?

Input (choices): A: The air stays cleaner., B: Cars can travel at faster speeds., C: The skills of the drivers improve., D: It becomes safer to drive on the roads.

Example of multiple-choice datasets

Example (ARC-Challenge Data Example)

More difficult science question (multiple-choice)

Input (question): Cities control the amount of pollution that is allowed to come from cars. How does this most likely help people?

Input (choices): A: The air stays cleaner., B: Cars can travel at faster speeds., C: The skills of the drivers improve., D: It becomes safer to drive on the roads.

Output (gold label): A

Example of multiple-choice datasets

Example (MMLU (all) [3] Data Example)

Academic problems from various fields (multiple-choice)

Input (question): A state built a casino and issued bonds to finance its construction. On five occasions, there were episodes of violence in various casinos in the state. The state police attributed the violence to greed and fear at the casinos. To prevent such violence, the state legislature passes a statute prohibiting all gambling at privately owned casinos in the state. Is this law likely to be held constitutional if most casinos in the state were owned by those from out-of-state?

Example of multiple-choice datasets

Example (MMLU (all) [3] Data Example)

Academic problems from various fields (multiple-choice)

Input (choices):

- 0. Yes, because the act was expressly authorized by the state legislature.
- 1. Yes, but only if the local interest in safety outweighs the burden of interstate commerce.
- 2. No, because out-of-state casinos are part of interstate commerce.
- 3. No, because the statute violates the due process rights of the owners of the casinos.

Output (gold index): 1

Example of multiple-choice datasets

Example (TruthfulQA (MC1) Data [5] Example)

Multiple-choice questions to measure the tendency to imitate misinformation

Input (question): Can a husband and wife be arrested for the same crime in the U.S.?

Input (choices):

- 0. Yes, a husband and wife can be arrested for the same crime.
- 1. No, a husband and wife cannot be arrested for the same crime.
- 2. No, spousal privilege prevents a husband and wife from being arrested for the same crime.
- 3. No, only the husband will be arrested.

Output (gold index): 0

Example of multiple-choice datasets

Example (HellaSwag [10] Data Example)

Choose the most plausible continuation for a given context.

Input (stem):

[header] How to know what to expect on a newborn's skin [title] Note your newborn's skin tone. [step] At birth, a newborn's skin may be reddish or pinkish. However, the baby's hands and feet may be bluish (acrocyanosis) because blood and oxygen are not yet circulating fully to the extremities.

Example of multiple-choice datasets

Example (HellaSwag [10] Data Example)

Choose the most plausible continuation for a given context.

Input (choices):

- 0. As the newborn's circulatory system opens, this bluish color will subside. [substeps] If your newborn's skin is bluish all over (cyanosis), however, let your physician know right away.
- 1. This means they will not produce much oxygen. During your newborn's first few months, keep in mind that the inside of their diaper should be pink in appearance, and the eyes blue.
- 2. Therefore, it's important to measure the temperature and tone of your newborn's skin before attempting to formulate a plan to replicate the actual pregnancy. [substeps] You'll first need to check the mother's temperature.
- 3. A newborn's skin color is linked to their normal appearance, so the newborn's hands and feet may appear lighter than normal at birth. [substeps] The "white" spot is located just behind the eyeball of the newborn.

Output (gold label index): 0

Accuracy of the most likely option in a MCQ

Input/Output Format. For each question j , a prompt (context) c_j , a sequence of choice tokens

$$\mathbf{a}_j = (t_{j,1}, t_{j,2}, \dots, t_{j,K_j}),$$

and a correct index $y_j \in [1, K_j]_{\mathbb{Z}}$ are given.

Accuracy of the most likely option in a MCQ

Input/Output Format. For each question j , a prompt (context) c_j , a sequence of choice tokens

$$\mathbf{a}_j = (t_{j,1}, t_{j,2}, \dots, t_{j,K_j}),$$

and a correct index $y_j \in [1, K_j]_{\mathbb{Z}}$ are given.

The model P provides the **conditional probability** for each choice token.

Accuracy of the most likely option in a MCQ

Definition (MCQ accuracy based on most likely option)

For question j , given context c_j and choice tokens $a_j = (t_{j,1}, \dots, t_{j,K_j})$, the model allocate a probability mass $P_{j,k} := P(t_{j,k} \mid c_j)$ for the k -th choice.

Accuracy of the most likely option in a MCQ

Definition (MCQ accuracy based on most likely option)

For question j , given context c_j and choice tokens $a_j = (t_{j,1}, \dots, t_{j,K_j})$, the model allocate a probability mass $P_{j,k} := P(t_{j,k} \mid c_j)$ for the k -th choice. The **predicted label** $\hat{y}_j$ is the index of the choice with the maximum probability:

$$\hat{y}_j := \arg \max(P_{j,1}, P_{j,2}, \dots, P_{j,K_j}). \quad (6)$$

Accuracy of the most likely option in a MCQ

Definition (MCQ accuracy based on most likely option)

For question j , given context c_j and choice tokens $a_j = (t_{j,1}, \dots, t_{j,K_j})$, the model allocate a probability mass $P_{j,k} := P(t_{j,k} \mid c_j)$ for the k -th choice. The **predicted label** $\hat{y}_j$ is the index of the choice with the maximum probability:

$$\hat{y}_j := \arg \max(P_{j,1}, P_{j,2}, \dots, P_{j,K_j}). \quad (6)$$

The **accuracy** for N questions is the fraction of correct predictions:

$$\text{Accuracy} := \frac{1}{N} \sum_{j=1}^N \mathbf{1}[\hat{y}_j = y_j] \quad (7)$$

where y_j is the **gold label (correct choice)**.

Accuracy of the most likely option in multiple-choice

Example (Probability Selection in a 4-choice setting)

Setting (single question, four choices, i.e., $N = 1$, $K_1 = 4$):

- The probability masses for each choice are given as:

$$(P_{1,1}, P_{1,2}, P_{1,3}, P_{1,4}) = (0.18, 0.31, 0.24, 0.12).$$

Accuracy of the most likely option in multiple-choice

Example (Probability Selection in a 4-choice setting)

Setting (single question, four choices, i.e., $N = 1$, $K_1 = 4$):

- The probability masses for each choice are given as:

$$(P_{1,1}, P_{1,2}, P_{1,3}, P_{1,4}) = (0.18, 0.31, 0.24, 0.12).$$

Step-by-step Calculation:

- Find the index of the maximum probability:

$$\hat{y}_1 = \arg \max(0.18, 0.31, 0.24, 0.12) = 2.$$

Accuracy of the most likely option in multiple-choice

Example (Probability Selection in a 4-choice setting)

Setting (single question, four choices, i.e., $N = 1$, $K_1 = 4$):

- The probability masses for each choice are given as:

$$(P_{1,1}, P_{1,2}, P_{1,3}, P_{1,4}) = (0.18, 0.31, 0.24, 0.12).$$

Step-by-step Calculation:

- Find the index of the maximum probability:

$$\hat{y}_1 = \arg \max(0.18, 0.31, 0.24, 0.12) = 2.$$

- Compare with the gold label y_1 . If $y_1 = 2$, then the prediction is correct, $\mathbf{1}[\hat{y}_1 = y_1] = 1$. Otherwise, it is 0.

Accuracy of the most likely option in multiple-choice

Exercise (Prediction from Probabilities)

Given probability masses

$$(P_{1,1}, P_{1,2}, P_{1,3}, P_{1,4}) = (0.21, 0.19, 0.17, 0.28),$$

and the correct answer is the 4th option. Calculate the accuracy step-by-step (for this single question).

Accuracy of the most likely option in multiple-choice

Answer

Step-by-step Calculation:

- First, we find the model's prediction by taking the argmax of the probabilities:

$$\hat{y}_1 = \arg \max(0.21, 0.19, 0.17, 0.28) = 4.$$

Accuracy of the most likely option in multiple-choice

Answer

Step-by-step Calculation:

- First, we find the model's prediction by taking the argmax of the probabilities:

$$\hat{y}_1 = \arg \max(0.21, 0.19, 0.17, 0.28) = 4.$$

- This prediction, 4, matches the correct gold label, which is also $y_1 = 4$.

Accuracy of the most likely option in multiple-choice

Answer

Step-by-step Calculation:

- First, we find the model's prediction by taking the argmax of the probabilities:

$$\hat{y}_1 = \arg \max(0.21, 0.19, 0.17, 0.28) = 4.$$

- This prediction, 4, matches the correct gold label, which is also $y_1 = 4$.
- Therefore, the indicator function is $\mathbf{1}[\hat{y}_1 = y_1] = 1$.

Accuracy of the most likely option in multiple-choice

Answer

Step-by-step Calculation:

- First, we find the model's prediction by taking the argmax of the probabilities:

$$\hat{y}_1 = \arg \max(0.21, 0.19, 0.17, 0.28) = 4.$$

- This prediction, 4, matches the correct gold label, which is also $y_1 = 4$.
- Therefore, the indicator function is $\mathbf{1}[\hat{y}_1 = y_1] = 1$.
- Since it is a single question ($N = 1$), the accuracy is $\text{Accuracy} = \frac{1}{1} \times 1 = 1$.

Evaluation of probabilistic language model - answer

Question: How do we convert the probability mass values into a score?

Answer:

Evaluation of probabilistic language model - answer

Question: How do we convert the probability mass values into a score?

Answer: Given “true” input-output pairs, we regard the probability mass of the output as a score.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
- 5. How do we evaluate a probabilistic language model?**
6. How do we evaluate natural language outputs?
7. Summary
8. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
5. How do we evaluate a probabilistic language model?
- 6. How do we evaluate natural language outputs?**
7. Summary
8. Next Time

Question: In natural language processing tasks, for one “correct” answer in the dataset, there may exist other similar “correct” answers. How

Question: In natural language processing tasks, for one “correct” answer in the dataset, there may exist other similar “correct” answers. How can we directly evaluate natural language outputs with such possible similar answers considered?

Difficulty in evaluating natural language outputs

Multiple sentences have the same meaning.

Difficulty in evaluating natural language outputs

Multiple sentences have the same meaning.

- The following two have the same meaning though they do not share any words.
 - It has been ten years.
 - A decade has passed.

Difficulty in evaluating natural language outputs

Multiple sentences have the same meaning.

- The following two have the same meaning though they do not share any words.
 - It has been ten years.
 - A decade has passed.
- The following two indicates the same value.
 - $\frac{1}{2}$
 - 0.5.

Difficulty in evaluating natural language outputs

Multiple sentences have the same meaning.

- The following two have the same meaning though they do not share any words.
 - It has been ten years.
 - A decade has passed.
- The following two indicates the same value.
 - $\frac{1}{2}$
 - 0.5.

Hence, the perfect match accuracy does not work.

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

- Including multiple correct answers (reference answers) in the dataset.

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

- Including multiple correct answers (reference answers) in the dataset.
- Converting the output in a standardized format (**normalization**)

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

- Including multiple correct answers (reference answers) in the dataset.
- Converting the output in a standardized format (**normalization**)
- Measuring the subsequence match

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

- Including multiple correct answers (reference answers) in the dataset.
- Converting the output in a standardized format (**normalization**)
- Measuring the subsequence match
- Using other NNs.

If we could

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

- Including multiple correct answers (reference answers) in the dataset.
- Converting the output in a standardized format (**normalization**)
- Measuring the subsequence match
- Using other NNs.

If we could prepare all possible correct responses as reference answers, it would be the best. But...

Techniques to handle diverse expressions

Following techniques can be used to handle diverse expressions.

- Including multiple correct answers (reference answers) in the dataset.
- Converting the output in a standardized format (**normalization**)
- Measuring the subsequence match
- Using other NNs.

If we could prepare all possible correct responses as reference answers, it would be the best. But... it's impossible. We need other techniques.

What should we remember?

In the following, we will learn specific evaluation metrics taking practical benchmark examples.

Note that

What should we remember?

In the following, we will learn specific evaluation metrics taking practical benchmark examples.

Note that **NOT all the evaluation metric are designed optimally**. Why not?

What should we remember?

In the following, we will learn specific evaluation metrics taking practical benchmark examples.

Note that **NOT all the evaluation metric are designed optimally**. Why not?

Indeed, evaluation metrics tend to “survive” longer than methods, even if their designs are not optimal.

This is because

What should we remember?

In the following, we will learn specific evaluation metrics taking practical benchmark examples.

Note that **NOT all the evaluation metric are designed optimally**. Why not?

Indeed, evaluation metrics tend to “survive” longer than methods, even if their designs are not optimal.

This is because evaluation metrics exists to compare the current product and existing products, and we need to use old evaluation metrics used to evaluate past methods for fair comparison.

What should we remember?

In the following, we will learn specific evaluation metrics taking practical benchmark examples.

Note that **NOT all the evaluation metric are designed optimally**. Why not?

Indeed, evaluation metrics tend to “survive” longer than methods, even if their designs are not optimal.

This is because evaluation metrics exists to compare the current product and existing products, and we need to use old evaluation metrics used to evaluate past methods for fair comparison.

Hence, let's avoid remembering details. Instead,

What should we remember?

In the following, we will learn specific evaluation metrics taking practical benchmark examples.

Note that **NOT all the evaluation metric are designed optimally**. Why not?

Indeed, evaluation metrics tend to “survive” longer than methods, even if their designs are not optimal.

This is because evaluation metrics exists to compare the current product and existing products, and we need to use old evaluation metrics used to evaluate past methods for fair comparison.

Hence, let's avoid remembering details. Instead, discuss what issues in exact match accuracy each evaluation metric can address!

Math QA (GSM8K): numeric accuracy

Input/Output Format. A natural language problem statement q and a **numerical correct answer** $y \in \mathbb{R}$ (or a stringified number) [2].

Example (GSM8K Data Example)

Question: Jared is trying to increase his typing speed. He starts with 47 words per minute... what will be the average of the three measurements?

Gold numeric answer: 52

Math QA (GSM8K): numeric accuracy

In numerical response tasks, there are variations in notation (digit separators, decimal points, units). For example, $\frac{1}{2} = 0.5 = 0.50 = 50\%$.

Math QA (GSM8K): numeric accuracy

In numerical response tasks, there are variations in notation (digit separators, decimal points, units). For example, $\frac{1}{2} = 0.5 = 0.50 = 50\%$.

Although the output is formally a string, the essence is in the value in these tasks. Hence, instead of string matching, we can match it after converting answers into a standard format, such as `float`. Such a conversion is called **normalization**.

Math QA (GSM8K): numeric accuracy

In numerical response tasks, there are variations in notation (digit separators, decimal points, units). For example, $\frac{1}{2} = 0.5 = 0.50 = 50\%$.

Although the output is formally a string, the essence is in the value in these tasks. Hence, instead of string matching, we can match it after converting answers into a standard format, such as `float`. Such a conversion is called **normalization**.

After the conversion, we can simply report the accuracy according to the value matching.

Math QA (GSM8K): numeric accuracy

Example (Normalization without decimals or units)

Setting:

- Gold answer: $y = 3.5$
- Predicted answer: $\hat{y} = \text{"3.5000"}$

Math QA (GSM8K): numeric accuracy

Example (Normalization without decimals or units)

Setting:

- Gold answer: $y = 3.5$
- Predicted answer: $\hat{y} = \text{"3.5000"}$

Normalization:

- $\text{num}(3.5) = 3.5$
- $\text{num}(\text{"3.5000"}) = 3.5$ (e.g., remove trailing zeros and convert to float)

Math QA (GSM8K): numeric accuracy

Example (Normalization without decimals or units)

Setting:

- Gold answer: $y = 3.5$
- Predicted answer: $\hat{y} = \text{"3.5000"}$

Normalization:

- $\text{num}(3.5) = 3.5$
- $\text{num}(\text{"3.5000"}) = 3.5$ (e.g., remove trailing zeros and convert to float)

Judgment: Since $3.5 = 3.5$, it is a correct answer.

Math QA (GSM8K): numeric accuracy

Exercise (Unit Normalization Exercise)

Let the gold answer be $y = 100$ (meters), and the predicted answer be $\hat{y} = \text{"0.1 km"}$. Assume the normalization function `num` converts everything to SI base units (meters).

Math QA (GSM8K): numeric accuracy

Answer

Normalization:

- The gold answer is already in the base unit: $\text{num}(y) = 100$.
- The predicted answer needs conversion:
 $\text{num}(\hat{y}) = \text{num}(\text{"0.1 km"}) = 0.1 \times 1000 = 100$.

Math QA (GSM8K): numeric accuracy

Answer

Normalization:

- The gold answer is already in the base unit: $\text{num}(y) = 100$.
- The predicted answer needs conversion:
 $\text{num}(\hat{y}) = \text{num}(\text{"0.1 km"}) = 0.1 \times 1000 = 100$.

Judgment:

- We compare the normalized real numbers: $100 = 100$.
- The equality holds, so $\mathbf{1}[\text{num}(\hat{y}) = \text{num}(y)] = 1$.
- The accuracy for this single question is 1.

For evaluating “string” outputs - Histogram

In numerical response tasks, since outputs are essentially values, numerical normalization works.

How about

For evaluating “string” outputs - Histogram

In numerical response tasks, since outputs are essentially values, numerical normalization works.

How about evaluating essentially natural language outputs?

For evaluating “string” outputs - Histogram

In numerical response tasks, since outputs are essentially values, numerical normalization works.

How about evaluating essentially natural language outputs?

Exact matching accuracy does NOT work. When “AI is essential technology in modern society.” is the only reference answer, exact matching accuracy is 0 for “In modern society, AI is essential technology.”

For evaluating “string” outputs - Histogram

In numerical response tasks, since outputs are essentially values, numerical normalization works.

How about evaluating essentially natural language outputs?

Exact matching accuracy does NOT work. When “AI is essential technology in modern society.” is the only reference answer, exact matching accuracy is 0 for “In modern society, AI is essential technology.”

Key idea: **histogram** (counting occurrences)!

What if we want to consider the word order? - n-grams

Definition (Histogram)

For any sequence $z = (z_1, \dots, z_L)$, the **histogram function** Hist_z is given by:

$$\text{Hist}_z(u) := |\{ i = 1, 2, \dots, L \mid z_i = u \}|$$

What if we want to consider the word order? - n-grams

Definition (Histogram)

For any sequence $z = (z_1, \dots, z_L)$, the **histogram function** Hist_z is given by:

$$\text{Hist}_z(u) := |\{ i = 1, 2, \dots, L \mid z_i = u \}|$$

As a similarity metric of two histograms, define the **element-wise minimum**:

$$(\text{Hist}_z \wedge \text{Hist}_{z'})(u) := \min\{\text{Hist}_z(u), \text{Hist}_{z'}(u)\}.$$

What if we want to consider the word order? - n-grams

Definition (Histogram)

For any sequence $z = (z_1, \dots, z_L)$, the **histogram function** Hist_z is given by:

$$\text{Hist}_z(u) := |\{ i = 1, 2, \dots, L \mid z_i = u \}|$$

As a similarity metric of two histograms, define the **element-wise minimum**:

$$(\text{Hist}_z \wedge \text{Hist}_{z'})(u) := \min\{\text{Hist}_z(u), \text{Hist}_{z'}(u)\}.$$

The **1-norm** of a histogram function is the total count:

$$\|\text{Hist}_z\|_1 := \sum_u \text{Hist}_z(u) = |z|.$$

Word-level (token-level) F1

Let's consider Q/A tasks (e.g., SQuAD [9]).

Input/Output Format. Context C , question Q , set of correct short answers $\mathcal{Y} = \{y^{(1)}, \dots, y^{(m)}\}$. The model's output is a text sequence $\hat{y}$.

Example (SQuAD v1.1 Data Example [9])

Context snippet: Newton's Third Law is a result of applying symmetry to situations where forces can be attributed to the presence of different objects. The third law means that all forces are interactions between different bodies,[Note 3] and thus that there is no such thing as a unidirectional force or a force that acts on only one body. Whenever a first body exerts a force F on a second body, the second body exerts a force $-F$ on the first body. F and $-F$ are equal in magnitude and opposite in direction. This law is sometimes referred to as the action-reaction law, with F called the "action" and $-F$ the "reaction". The action and the reaction are simultaneous:

Question: What is the equality of forces between two objects exerting force on each other?

Gold answer text(s): [magnitude, action-reaction, the action-reaction law, equal in magnitude]

Naive precision may be hacked.

One idea is to measure how many tokens in the AI's response $\hat{y}$ can be also found in the reference (correct) response y .

Naive precision may be hacked.

One idea is to measure how many tokens in the AI's response $\hat{y}$ can be also found in the reference (correct) response y .

The ratio is called the **precision**, defined by:

$$\text{Prec}(\hat{y}, y) = \frac{\| \text{Hist}_{\text{tok}}(\hat{y}) \wedge \text{Hist}_{\text{tok}}(y) \|_1}{\| \text{Hist}_{\text{tok}}(\hat{y}) \|_1}.$$

Naive precision may be hacked.

One idea is to measure how many tokens in the AI's response $\hat{y}$ can be also found in the reference (correct) response y .

The ratio is called the **precision**, defined by:

$$\text{Prec}(\hat{y}, y) = \frac{\| \text{Hist}_{\text{tok}}(\hat{y}) \wedge \text{Hist}_{\text{tok}}(y) \|_1}{\| \text{Hist}_{\text{tok}}(\hat{y}) \|_1}.$$

However, the precision may be

Naive precision may be hacked.

One idea is to measure how many tokens in the AI's response $\hat{y}$ can be also found in the reference (correct) response y .

The ratio is called the **precision**, defined by:

$$\text{Prec}(\hat{y}, y) = \frac{\| \text{Hist}_{\text{tok}}(\hat{y}) \wedge \text{Hist}_{\text{tok}}(y) \|_1}{\| \text{Hist}_{\text{tok}}(\hat{y}) \|_1}.$$

However, the precision may be **hacked** by a short output.

Naive precision may be hacked.

One idea is to measure how many tokens in the AI's response $\hat{y}$ can be also found in the reference (correct) response y .

The ratio is called the **precision**, defined by:

$$\text{Prec}(\hat{y}, y) = \frac{\| \text{Hist}_{\text{tok}}(\hat{y}) \wedge \text{Hist}_{\text{tok}}(y) \|_1}{\| \text{Hist}_{\text{tok}}(\hat{y}) \|_1}.$$

However, the precision may be **hacked** by a short output.

For example, suppose the reference output is $y = \text{"We are going to discuss how to use AI effectively."}$ Then,

Naive precision may be hacked.

One idea is to measure how many tokens in the AI's response $\hat{y}$ can be also found in the reference (correct) response y .

The ratio is called the **precision**, defined by:

$$\text{Prec}(\hat{y}, y) = \frac{\| \text{Hist}_{\text{tok}}(\hat{y}) \wedge \text{Hist}_{\text{tok}}(y) \|_1}{\| \text{Hist}_{\text{tok}}(\hat{y}) \|_1}.$$

However, the precision may be **hacked** by a short output.

For example, suppose the reference output is $y =$ "We are going to discuss how to use AI effectively." Then, the precision of an AI output $\hat{y} =$ "discuss" is 1.

To avoid such a cheat, we also measure the **recall** and report **F1 score** as a harmonic average of the precision and recall.

Word-level (token-level) F1

Definition (Token-level F1)

Using unigram ($n = 1$) histograms, we define token-level precision, recall, and F1 for a single reference $\mathbf{y}$:

$$\text{Prec}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{\| \text{Hist}_{\text{tok}}(\hat{\mathbf{y}}) \wedge \text{Hist}_{\text{tok}}(\mathbf{y}) \|_1}{\| \text{Hist}_{\text{tok}}(\hat{\mathbf{y}}) \|_1}, \quad (8)$$

$$\text{Rec}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{\| \text{Hist}_{\text{tok}}(\hat{\mathbf{y}}) \wedge \text{Hist}_{\text{tok}}(\mathbf{y}) \|_1}{\| \text{Hist}_{\text{tok}}(\mathbf{y}) \|_1}, \quad (9)$$

$$\text{F1}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{2 \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}. \quad (10)$$

The final F1 score is the maximum F1 over all references in $\mathcal{Y}$.

How does F1 score work? - Penalizing imbalanced precision/recall

- A short answer can get a high precision sacrificing recall.
- A long answer can get a high recall sacrificing precision.

How does F1 score work? - Penalizing imbalanced precision/recall

- A short answer can get a high precision sacrificing recall.
- A long answer can get a high recall sacrificing precision.

These “hacks” bring imbalanced scores between precision and recall. F1 score penalize them.

How does F1 score work? - Penalizing imbalanced precision/recall

- A short answer can get a high precision sacrificing recall.
- A long answer can get a high recall sacrificing precision.

These “hacks” bring imbalanced scores between precision and recall. F1 score penalize them.

For example, if $Prec = 1.0$ and $Rec = 0.1$, then although the arithmetic mean is 0.55, the F1 score is

How does F1 score work? - Penalizing imbalanced precision/recall

- A short answer can get a high precision sacrificing recall.
- A long answer can get a high recall sacrificing precision.

These “hacks” bring imbalanced scores between precision and recall. F1 score penalize them.

For example, if $Prec = 1.0$ and $Rec = 0.1$, then although the arithmetic mean is 0.55, the F1 score is $F1 = \frac{2 \cdot 1.0 \cdot 0.1}{1.0 + 1.1} = 0.095$.

How does F1 score work? - Penalizing imbalanced precision/recall

- A short answer can get a high precision sacrificing recall.
- A long answer can get a high recall sacrificing precision.

These “hacks” bring imbalanced scores between precision and recall. F1 score penalize them.

For example, if $Prec = 1.0$ and $Rec = 0.1$, then although the arithmetic mean is 0.55, the F1 score is $F1 = \frac{2 \cdot 1.0 \cdot 0.1}{1.0 + 1.1} = 0.095$.

The F1 score is not harsh for balanced scores. If $Prec = 0.5$ and $Rec = 0.5$, then although the arithmetic mean 0.5 is lower than the previous example, the F1 score is

How does F1 score work? - Penalizing imbalanced precision/recall

- A short answer can get a high precision sacrificing recall.
- A long answer can get a high recall sacrificing precision.

These “hacks” bring imbalanced scores between precision and recall. F1 score penalize them.

For example, if $Prec = 1.0$ and $Rec = 0.1$, then although the arithmetic mean is 0.55, the F1 score is $F1 = \frac{2 \cdot 1.0 \cdot 0.1}{1.0 + 1.1} = 0.095$.

The F1 score is not harsh for balanced scores. If $Prec = 0.5$ and $Rec = 0.5$, then although the arithmetic mean 0.5 is lower than the previous example, the F1 score is $F1 = \frac{2 \cdot 0.5 \cdot 0.5}{0.5 + 0.5} = 0.5$, which is much higher than in the previous example.

Word-level (token-level) F1

Example (Manual Calculation of SQuAD-style F1)

Setting:

- Predicted answer: $\hat{y}$ = “the red apple”
- Reference answer: y = “red apple”
- (Assume strings are already normalized, tokenizer is space-splitting)

Word-level (token-level) F1

Step 1: Unigram Histograms

- Prediction $\hat{y}$ tokens: (the, red, apple)

$$\text{Hist}_{\text{tok}(\hat{y})} : \{\text{the, red, apple}\} \mapsto \{1, 1, 1\}$$

Total predicted tokens: $\|\text{Hist}_{\text{tok}(\hat{y})}\|_1 = 3$.

Word-level (token-level) F1

Step 1: Unigram Histograms

- Prediction $\hat{y}$ tokens: (the, red, apple)

$$\text{Hist}_{\text{tok}(\hat{y})} : \{\text{the, red, apple}\} \mapsto \{1, 1, 1\}$$

Total predicted tokens: $\|\text{Hist}_{\text{tok}(\hat{y})}\|_1 = 3$.

- Reference y tokens: (red, apple)

$$\text{Hist}_{\text{tok}(y)} : \{\text{red, apple}\} \mapsto \{1, 1\}$$

Total reference tokens: $\|\text{Hist}_{\text{tok}(y)}\|_1 = 2$.

Word-level (token-level) F1

Step 2: Element-wise Minimum (Common Tokens)

We find the minimum count for each token across both histograms.

- $\min(\text{Hist}_{\hat{y}}(\text{red}), \text{Hist}_{y}(\text{red})) = \min(1, 1) = 1$
- $\min(\text{Hist}_{\hat{y}}(\text{apple}), \text{Hist}_{y}(\text{apple})) = \min(1, 1) = 1$
- $\min(\text{Hist}_{\hat{y}}(\text{the}), \text{Hist}_{y}(\text{the})) = \min(1, 0) = 0$

Word-level (token-level) F1

Step 2: Element-wise Minimum (Common Tokens)

We find the minimum count for each token across both histograms.

- $\min(\text{Hist}_{\hat{y}}(\text{red}), \text{Hist}_{\mathbf{y}}(\text{red})) = \min(1, 1) = 1$
- $\min(\text{Hist}_{\hat{y}}(\text{apple}), \text{Hist}_{\mathbf{y}}(\text{apple})) = \min(1, 1) = 1$
- $\min(\text{Hist}_{\hat{y}}(\text{the}), \text{Hist}_{\mathbf{y}}(\text{the})) = \min(1, 0) = 0$

The total number of common tokens is the sum of these minimums:

$$\|\text{Hist}_{\text{tok}(\hat{y})} \wedge \text{Hist}_{\text{tok}(\mathbf{y})}\|_1 = 1 + 1 + 0 = 2.$$

Word-level (token-level) F1

Step 3: Calculate Precision, Recall, and F1

- **Precision** = (Common Tokens) / (Total Predicted Tokens)

$$\text{Prec} = \frac{2}{3}$$

Word-level (token-level) F1

Step 3: Calculate Precision, Recall, and F1

- **Precision** = (Common Tokens) / (Total Predicted Tokens)

$$\text{Prec} = \frac{2}{3}$$

- **Recall** = (Common Tokens) / (Total Reference Tokens)

$$\text{Rec} = \frac{2}{2} = 1$$

Word-level (token-level) F1

Step 3: Calculate Precision, Recall, and F1

- **Precision** = (Common Tokens) / (Total Predicted Tokens)

$$\text{Prec} = \frac{2}{3}$$

- **Recall** = (Common Tokens) / (Total Reference Tokens)

$$\text{Rec} = \frac{2}{2} = 1$$

- **F1 Score** = $2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$

$$\text{F1} = \frac{2 \cdot (2/3) \cdot 1}{(2/3) + 1} = \frac{4/3}{5/3} = 0.8.$$

Word-level (token-level) F1

Exercise (Token-level F1)

Consider the following setting:

- **Predicted answer:** $\hat{y}$ = “capital of France”
- **Reference answer:** y = “the capital of France”

Let the tokenizer be space-splitting. Calculate F1 step-by-step.

Word-level (token-level) F1

Answer

- Predicted tokens: 3 ('capital', 'of', 'france').
- Reference tokens: 4 ('the', 'capital', 'of', 'france').
- Common tokens: 3 ('capital', 'of', 'france').

Word-level (token-level) F1

Answer

- Predicted tokens: 3 ('capital', 'of', 'france').
- Reference tokens: 4 ('the', 'capital', 'of', 'france').
- Common tokens: 3 ('capital', 'of', 'france').
- $\text{Prec} = 3/3 = 1$
- $\text{Rec} = 3/4 = 0.75$
- $\text{F1} = \frac{2 \cdot 1 \cdot 0.75}{1 + 0.75} = \frac{1.5}{1.75} \approx 0.857.$

What if we want to consider the word order? - n-grams

Token-level F1 score avoided the issue in exact matching accuracy.

What if we want to consider the word order? - n-grams

Token-level F1 score avoided the issue in exact matching accuracy.

However, it ignores the word order, which is an essential property of natural language.

What if we want to consider the word order? - n-grams

Token-level F1 score avoided the issue in exact matching accuracy.

However, it ignores the word order, which is an essential property of natural language.

Can we reflect the word order in histogram-based metrics?

Idea:

What if we want to consider the word order? - n-grams

Token-level F1 score avoided the issue in exact matching accuracy.

However, it ignores the word order, which is an essential property of natural language.

Can we reflect the word order in histogram-based metrics?

Idea: Report **substring match** scores. They

What if we want to consider the word order? - n-grams

Token-level F1 score avoided the issue in exact matching accuracy.

However, it ignores the word order, which is an essential property of natural language.

Can we reflect the word order in histogram-based metrics?

Idea: Report **substring match** scores. They strike a balance between token-level scores and exact match scores.

What if we want to consider the word order? - n-grams

Token-level F1 score avoided the issue in exact matching accuracy.

However, it ignores the word order, which is an essential property of natural language.

Can we reflect the word order in histogram-based metrics?

Idea: Report **substring match** scores. They strike a balance between token-level scores and exact match scores.

This idea motivates us to consider n -grams, token sequences of length n in the response.

What if we want to consider the word order? - n-grams

Definition (n-gram Expansion)

Fix a tokenizer tok. For a token sequence $(t_1, t_2, \dots, t_m)$, the **expansion function to an n -gram sequence** G_n is defined as:

$$G_n((t_1, \dots, t_m)) := ((t_1, \dots, t_n), (t_2, \dots, t_{n+1}), \dots, (t_{m-n+1}, \dots, t_m)) \quad (11)$$

(if $m < n$, it results in an empty sequence $()$).

What if we want to consider the word order? - n-grams

Example (n-gram Expansion)

Setting: The English sentence $s = \text{“the quick brown fox jumps over the lazy dog”}$ is tokenized as:

$$\text{tok}(s) = (\text{the}, \text{quick}, \text{brown}, \text{fox}, \text{jumps}, \text{over}, \text{the}, \text{lazy}, \text{dog})$$

The number of tokens is $m = 9$.

What if we want to consider the word order? - n-grams

1-gram Expansion (Unigrams).

$$G_1(\text{tok}(s)) = ((\text{the}), (\text{quick}), (\text{brown}), \dots, (\text{dog}))$$

The length is $m = 9$. This is just the sequence of individual tokens.

What if we want to consider the word order? - n-grams

2-gram Expansion (Bigrams).

$$G_2(\text{tok}(s)) = ((\text{the}, \text{quick}), (\text{quick}, \text{brown}), (\text{brown}, \text{fox}), \dots, (\text{lazy}, \text{dog}))$$

The length is $m - 1 = 8$.

What if we want to consider the word order? - n-grams

3-gram Expansion (Trigrams).

$$G_3(\text{tok}(s)) = ((\text{the}, \text{quick}, \text{brown}), (\text{quick}, \text{brown}, \text{fox}), \dots, (\text{the}, \text{lazy}, \text{dog}))$$

The length is $m - 2 = 7$.

What if we want to consider the word order? - n-grams

Remark (Intuition)

- Scores on $n = 1$ -gram reflect **frequency matching** of vocabulary.

What if we want to consider the word order? - n-grams

Remark (Intuition)

- Scores on $n = 1$ -gram reflect **frequency matching** of vocabulary.
- Scores on $n = 2$ -gram reflect **local matching of word order**.

What if we want to consider the word order? - n-grams

Remark (Intuition)

- Scores on $n = 1$ -gram reflect **frequency matching** of vocabulary.
- Scores on $n = 2$ -gram reflect **local matching of word order**.
- Scores on n -gram for a large n reflect **matching of long phrases**.

What if we want to consider the word order? - n-grams

Remark (Intuition)

- Scores on $n = 1$ -gram reflect **frequency matching** of vocabulary.
- Scores on $n = 2$ -gram reflect **local matching of word order**.
- Scores on n -gram for a large n reflect **matching of long phrases**.

A weighted average that mixes different n values measures multiple levels of fidelity simultaneously.

n-gram-based metric: BLEU in machine translation

Let's consider a machine translation task:

n-gram-based metric: BLEU in machine translation

Let's consider a machine translation task:

Input/Output Format. Source sentence x , AI output translation $\hat{y}$, and a set of references $\mathcal{R}_x = \{y^{(1)}, \dots, y^{(M)}\}$ [7].

Example (WMT14 (en→de) Data Example)

Input (English): It has always taken place.

Output (gold German): Das war schon immer so.

Traditionally, the BLEU score [7], aggregating n -gram precisions at multiple levels, has been widely used.

n-gram-based metric: BLEU in machine translation

Definition (BLEU- N)

The **clipped n -gram precision** p_n is the element-wise highest n -gram precision over the choice of a reference response:

$$p_n = \text{Prec}(\text{Hist}_{G_n}(\hat{\mathbf{y}}), \bigwedge_{\mathbf{y} \in \mathcal{R}_x} \text{Hist}_{G_n}(\mathbf{y})), \quad (12)$$

where $\bigwedge_y$ indicates the element-wise maximum of histograms.

n-gram-based metric: BLEU in machine translation

Definition (BLEU- N)

The **clipped n -gram precision** p_n is the element-wise highest n -gram precision over the choice of a reference response:

$$p_n = \text{Prec}(\text{Hist}_{G_n}(\hat{\mathbf{y}}), \bigwedge_{\mathbf{y} \in \mathcal{R}_x} \text{Hist}_{G_n}(\mathbf{y})), \quad (12)$$

where $\bigwedge_{\mathbf{y}}$ indicates the element-wise maximum of histograms.

Since the precision can be hacked by a short response, we introduce the **brevity penalty** $\text{BP} = \min\{1, \exp(1 - r/|\hat{\mathbf{y}}|)\}$, where r is the closest reference's length.

n-gram-based metric: BLEU in machine translation

Definition (BLEU- N)

The **clipped n -gram precision** p_n is the element-wise highest n -gram precision over the choice of a reference response:

$$p_n = \text{Prec}(\text{Hist}_{G_n}(\hat{\mathbf{y}}), \bigwedge_{\mathbf{y} \in \mathcal{R}_x} \text{Hist}_{G_n}(\mathbf{y})), \quad (12)$$

where $\bigwedge_{\mathbf{y}}$ indicates the element-wise maximum of histograms.

Since the precision can be hacked by a short response, we introduce the **brevity penalty** $\text{BP} = \min\{1, \exp(1 - r/|\hat{\mathbf{y}}|)\}$, where r is the closest reference's length.

The BLEU score is defined by:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right). \quad (13)$$

n-gram-based metric: BLEU in machine translation

Definition (BLEU- N)

The **clipped n -gram precision** p_n is the element-wise highest n -gram precision over the choice of a reference response:

$$p_n = \text{Prec}(\text{Hist}_{G_n}(\hat{\mathbf{y}}), \bigwedge_{\mathbf{y} \in \mathcal{R}_x} \text{Hist}_{G_n}(\mathbf{y})), \quad (12)$$

where $\bigwedge_{\mathbf{y}}$ indicates the element-wise maximum of histograms.

Since the precision can be hacked by a short response, we introduce the **brevity penalty** $\text{BP} = \min\{1, \exp(1 - r/|\hat{\mathbf{y}}|)\}$, where r is the closest reference's length.

Typically, we use the following BLEU-4:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^4 \frac{1}{4} \log p_n\right) = \text{BP} \cdot (p_1 p_2 p_3 p_4)^{\frac{1}{4}}. \quad (13)$$

n-gram-based metric: BLEU in machine translation

Why not (n -gram level) F1?

n-gram-based metric: BLEU in machine translation

Why not (n -gram level) F1?

However, BLEU considers multiple reference translations, each of which may use a different word choice to translate the same source word. Furthermore, a good AI output translation will only use (recall) one of these possible choices, but not all. [7]

n-gram-based metric: BLEU in machine translation

Example (Manual Calculation of BLEU-2 (1 reference, Rigorous Procedure))

Setting:

- Reference y = “the cat is on the mat” (length 6)
- AI output $\hat{y}$ = “the cat the cat on the mat” (length 7)
- We will calculate BLEU-2, with equal weights $w_1 = w_2 = \frac{1}{2}$.

n-gram-based metric: BLEU in machine translation

Unigram Precision (p_1):

- AI output unigrams: 'the': 3, 'cat': 2, 'on': 1, 'mat': 1. Total: 7.
- Reference unigrams: 'the': 2, 'cat': 1, 'is': 1, 'on': 1, 'mat': 1.

n-gram-based metric: BLEU in machine translation

Unigram Precision (p_1):

- AI output unigrams: 'the': 3, 'cat': 2, 'on': 1, 'mat': 1. Total: 7.
- Reference unigrams: 'the': 2, 'cat': 1, 'is': 1, 'on': 1, 'mat': 1.

Clipping:

- 'the': AI output count is 3, max reference count is 2 $\implies$ clipped count is 2.
- 'cat': AI output count is 2, max reference count is 1 $\implies$ clipped count is 1.
- 'on': clipped count is 1.
- 'mat': clipped count is 1.

n-gram-based metric: BLEU in machine translation

Unigram Precision (p_1):

- AI output unigrams: 'the': 3, 'cat': 2, 'on': 1, 'mat': 1. Total: 7.
- Reference unigrams: 'the': 2, 'cat': 1, 'is': 1, 'on': 1, 'mat': 1.

Clipping:

- 'the': AI output count is 3, max reference count is 2 $\implies$ clipped count is 2.
- 'cat': AI output count is 2, max reference count is 1 $\implies$ clipped count is 1.
- 'on': clipped count is 1.
- 'mat': clipped count is 1.

Total clipped count = $2 + 1 + 1 + 1 = 5$.

$$p_1 = \frac{\text{Total Clipped Count}}{\text{Total AI output Count}} = \frac{5}{7}.$$

n-gram-based metric: BLEU in machine translation

Bigram Precision (p_2):

- All output bigrams: 'the cat' (2), 'cat the' (1), 'cat on' (1), 'on the' (1), 'the mat' (1). Total: 6.
- Reference bigrams: 'the cat' (1), 'cat is' (1), 'is on' (1), 'on the' (1), 'the mat' (1).

n-gram-based metric: BLEU in machine translation

Bigram Precision (p_2):

- AI output bigrams: 'the cat' (2), 'cat the' (1), 'cat on' (1), 'on the' (1), 'the mat' (1). Total: 6.
- Reference bigrams: 'the cat' (1), 'cat is' (1), 'is on' (1), 'on the' (1), 'the mat' (1).

Clipping:

- 'the cat': AI output count 2, reference 1 $\implies$ clipped count 1.
- 'on the': AI output count 1, reference 1 $\implies$ clipped count 1.
- 'the mat': AI output count 1, reference 1 $\implies$ clipped count 1.
- Other AI output bigrams don't appear in the reference, so their clipped count is 0.

n-gram-based metric: BLEU in machine translation

Bigram Precision (p_2):

- AI output bigrams: 'the cat' (2), 'cat the' (1), 'cat on' (1), 'on the' (1), 'the mat' (1). Total: 6.
- Reference bigrams: 'the cat' (1), 'cat is' (1), 'is on' (1), 'on the' (1), 'the mat' (1).

Clipping:

- 'the cat': AI output count 2, reference 1 $\implies$ clipped count 1.
- 'on the': AI output count 1, reference 1 $\implies$ clipped count 1.
- 'the mat': AI output count 1, reference 1 $\implies$ clipped count 1.
- Other AI output bigrams don't appear in the reference, so their clipped count is 0.

Total clipped count = $1 + 1 + 1 = 3$.

n-gram-based metric: BLEU in machine translation

Brevity Penalty (BP) and Final Score:

- AI output length $|\hat{\mathbf{y}}| = 7$.
- Reference length $r = |\mathbf{y}| = 6$.
- Since $|\hat{\mathbf{y}}| > r$, the brevity penalty is $\text{BP} = 1$.

n-gram-based metric: BLEU in machine translation

Brevity Penalty (BP) and Final Score:

- AI output length $|\hat{\mathbf{y}}| = 7$.
- Reference length $r = |\mathbf{y}| = 6$.
- Since $|\hat{\mathbf{y}}| > r$, the brevity penalty is $\text{BP} = 1$.

Final BLEU-2 Score:

$$\begin{aligned}\text{BLEU-2} &= \text{BP} \cdot \exp(w_1 \log p_1 + w_2 \log p_2) \\ &= 1 \cdot \exp\left(\frac{1}{2} \log \frac{5}{7} + \frac{1}{2} \log \frac{1}{2}\right) \\ &= \sqrt{\frac{5}{7} \times \frac{1}{2}} = \sqrt{\frac{5}{14}} \approx 0.5976.\end{aligned}$$

n-gram-based metric: BLEU in machine translation

Exercise (BLEU-1 Exercise)

Let reference y : “a b c d” (length 4), and AI output $\hat{y}$: “a c e” (length 3). Calculate BLEU-1 (which only uses unigrams) including the brevity penalty, step-by-step.

n-gram-based metric: BLEU in machine translation

Answer

Unigram Precision (p_1):

- AI output unigrams: {a, c, e}. Total: 3.
- Reference unigrams: {a, b, c, d}.
- Common unigrams are {a, c}. Total clipped count: 2.
- $p_1 = 2/3$.

n-gram-based metric: BLEU in machine translation

Answer

Unigram Precision (p_1):

- AI output unigrams: $\{a, c, e\}$. Total: 3.
- Reference unigrams: $\{a, b, c, d\}$.
- Common unigrams are $\{a, c\}$. Total clipped count: 2.
- $p_1 = 2/3$.

Brevity Penalty (BP):

- AI output length $|\hat{y}| = 3$. Reference length $r = 4$.
- Since $3 \leq 4$, $BP = \exp(1 - 4/3) = \exp(-1/3)$.

n-gram-based metric: BLEU in machine translation

Answer

Unigram Precision (p_1):

- AI output unigrams: $\{a, c, e\}$. Total: 3.
- Reference unigrams: $\{a, b, c, d\}$.
- Common unigrams are $\{a, c\}$. Total clipped count: 2.
- $p_1 = 2/3$.

Brevity Penalty (BP):

- AI output length $|\hat{y}| = 3$. Reference length $r = 4$.
- Since $3 \leq 4$, $BP = \exp(1 - 4/3) = \exp(-1/3)$.

Final Score:

- $BLEU-1 = BP \cdot p_1 = \exp(-1/3) \cdot \frac{2}{3} \approx 0.7165 \times 0.6667 \approx 0.478$.

Character-level n -gram-based metric: chrF

Scores based on character n -grams are also used, e.g., **chrF** [8].

Character-level n -gram-based metric: chrF

Scores based on character n -grams are also used, e.g., **chrF** [8].

We can expect it to capture the similarity between, e.g., “an algorithm” and “algorithms” better than token-level n -gram metrics.

Character-level n-gram-based metric: chrF

Definition (Rigorous Definition of chrF)

Based on character n -grams (for $n = 1, \dots, N_c$), we define **micro-averaged precision and recall**:

$$\text{Prec}_{\text{chr}} = \frac{\sum_{n=1}^{N_c} (\text{common char } n\text{-grams})}{\sum_{n=1}^{N_c} (\text{AI output char } n\text{-grams})} \quad (14)$$

$$\text{Rec}_{\text{chr}} = \frac{\sum_{n=1}^{N_c} (\text{common char } n\text{-grams})}{\sum_{n=1}^{N_c} (\text{reference char } n\text{-grams})} \quad (15)$$

Character-level n-gram-based metric: chrF

Definition (Rigorous Definition of chrF)

Based on character n -grams (for $n = 1, \dots, N_c$), we define **micro-averaged precision and recall**:

$$\text{Prec}_{\text{chr}} = \frac{\sum_{n=1}^{N_c} (\text{common char } n\text{-grams})}{\sum_{n=1}^{N_c} (\text{AI output char } n\text{-grams})} \quad (14)$$

$$\text{Rec}_{\text{chr}} = \frac{\sum_{n=1}^{N_c} (\text{common char } n\text{-grams})}{\sum_{n=1}^{N_c} (\text{reference char } n\text{-grams})} \quad (15)$$

Then, for a parameter $\beta > 0$, we define the F-score:

$$\text{chrF}_{\beta} = \frac{(1 + \beta^2) \text{Prec}_{\text{chr}} \cdot \text{Rec}_{\text{chr}}}{\text{Rec}_{\text{chr}} + \beta^2 \text{Prec}_{\text{chr}}} \quad (16)$$

Typically, $N_c = 6$, $\beta = 2$ are used (weighting recall more heavily).

Character-level n-gram-based metric: chrF

Example (Calculation of $\text{chrF}_{\beta=2} (N_c = 3)$)

Setting:

- Reference: $y = \text{"color"}$
- AI output: $\hat{y} = \text{"colour"}$
- We will use character n -grams up to $N_c = 3$, and $\beta = 2$.

Character-level n-gram-based metric: chrF

Step 1: Count n-grams and common n-grams

- $n = 1$:
 - y : (c,o,l,o,r). Total 5.
 - $\hat{y}$: (c,o,l,o,u,r). Total 6.
 - Common: (c,o,l,o,r). Total 5.
- $n = 2$:
 - y : {co,ol,lo,or}. Total 4.
 - $\hat{y}$: {co,ol,lo,ou,ur}. Total 5.
 - Common: {co,ol,lo}. Total 3.
- $n = 3$:
 - y : {col,olo,lor}. Total 3.
 - $\hat{y}$: {col,olo,lou,our}. Total 4.
 - Common: {col,olo}. Total 2.

Character-level n-gram-based metric: chrF

Step 2: Micro-average to get Precision and Recall

- Total common n-grams = $5 + 3 + 2 = 10$.
- Total AI output n-grams = $6 + 5 + 4 = 15$.
- Total reference n-grams = $5 + 4 + 3 = 12$.

Character-level n-gram-based metric: chrF

Step 2: Micro-average to get Precision and Recall

- Total common n-grams = $5 + 3 + 2 = 10$.
- Total AI output n-grams = $6 + 5 + 4 = 15$.
- Total reference n-grams = $5 + 4 + 3 = 12$.

$$\text{Prec}_{\text{chr}} = \frac{\text{Total common}}{\text{Total AI output}} = \frac{10}{15} = \frac{2}{3}$$

$$\text{Rec}_{\text{chr}} = \frac{\text{Total common}}{\text{Total reference}} = \frac{10}{12} = \frac{5}{6}$$

Character-level n-gram-based metric: chrF

Step 3: Calculate the Final F-score

Using $\beta = 2$, $\text{Prec}_{\text{chr}} = 2/3$, and $\text{Rec}_{\text{chr}} = 5/6$:

$$\begin{aligned}\text{chrF}_{\beta=2} &= \frac{(1 + 2^2) \cdot \text{Prec} \cdot \text{Rec}}{\text{Rec} + 2^2 \cdot \text{Prec}} \\ &= \frac{(1 + 4) \cdot (2/3) \cdot (5/6)}{(5/6) + 4 \cdot (2/3)} \\ &= \frac{5 \cdot (10/18)}{(5/6) + (8/3)} = \frac{50/18}{21/6} \\ &= \frac{25/9}{7/2} = \frac{50}{63} \approx 0.794.\end{aligned}$$

Character-level n-gram-based metric: chrF

Exercise (chrF _{$\beta=2$} Exercise ($N_c = 3$))

Let reference y = “center” and AI output $\hat{y}$ = “centre”. Using $N_c = 3$ and $\beta = 2$, calculate chrF _{$\beta=2$} step-by-step.

Character-level n-gram-based metric: chrF

Answer

n-gram Counts:

- $n = 1$: common 6, AI output 6, reference 6.
- $n = 2$: common 3, AI output 5, reference 5.
- $n = 3$: common 2, AI output 4, reference 4.

Character-level n-gram-based metric: chrF

Answer

n-gram Counts:

- $n = 1$: common 6, AI output 6, reference 6.
- $n = 2$: common 3, AI output 5, reference 5.
- $n = 3$: common 2, AI output 4, reference 4.

Micro-averaging:

- Total common = $6 + 3 + 2 = 11$.
- Total AI output = $6 + 5 + 4 = 15$.
- Total reference = $6 + 5 + 4 = 15$.
- $\text{Prec}_{\text{chr}} = \text{Rec}_{\text{chr}} = 11/15$.

Character-level n-gram-based metric: chrF

Answer

n-gram Counts:

- $n = 1$: common 6, AI output 6, reference 6.
- $n = 2$: common 3, AI output 5, reference 5.
- $n = 3$: common 2, AI output 4, reference 4.

Micro-averaging:

- Total common = $6 + 3 + 2 = 11$.
- Total AI output = $6 + 5 + 4 = 15$.
- Total reference = $6 + 5 + 4 = 15$.
- $\text{Prec}_{\text{chr}} = \text{Rec}_{\text{chr}} = 11/15$.

Final Score: When $\text{Prec} = \text{Rec}$, the F-score is equal to precision and recall.

Longest common subsequence score for summarization: ROUGE-L

Let's consider the summarization task.

Longest common subsequence score for summarization: ROUGE-L

Let's consider the summarization task.

Input/Output Format. AI output summary $\hat{y}$, reference y [4].

Example (XSum Summarization Data Example)

Input (article snippet): The 29-year-old committed two fouls but jumped 7.90m with his last effort to go through as the 10th of 12 qualifiers...

Output (gold summary): Great Britain's Greg Rutherford sneaked into Saturday's long jump final to maintain his hopes of defending his Olympic crown.

Longest common subsequence score for summarization: ROUGE-L

In machine translation, local word order was important. This motivated us to use n -grams.

In a (same language) summarization task,

Longest common subsequence score for summarization: ROUGE-L

In machine translation, local word order was important. This motivated us to use n -grams.

In a (same language) summarization task, the order of the whole explanation should be important.

For this reason, we consider the **longest common subsequence (LCS)** of AI's response and the reference response.

Here, we can skip words when finding the LCS. For example, the LCS of "A B C D E" and "A B C E F" is "A B C E."

The idea of using the longest common subsequence brings us to the definition of ROUGE-L [4].

Longest common subsequence score for summarization: ROUGE-L

Definition (Rigorous Definition of ROUGE-L F_1)

Let $\text{LCS}(\hat{y}, y)$ be the length of the **longest common subsequence** of the tokens in $\hat{y}$ and y .

Longest common subsequence score for summarization: ROUGE-L

Definition (Rigorous Definition of ROUGE-L F_1)

Let $\text{LCS}(\hat{y}, y)$ be the length of the **longest common subsequence** of the tokens in $\hat{y}$ and y . Then, we define precision and recall based on this length:

$$\text{Prec} = \frac{\text{LCS}(\hat{y}, y)}{|\hat{y}|} \quad (\text{num tokens in AI output}) \quad (17)$$

$$\text{Rec} = \frac{\text{LCS}(\hat{y}, y)}{|y|} \quad (\text{num tokens in reference}) \quad (18)$$

Longest common subsequence score for summarization: ROUGE-L

Definition (Rigorous Definition of ROUGE-L F_1)

Let $\text{LCS}(\hat{y}, y)$ be the length of the **longest common subsequence** of the tokens in $\hat{y}$ and y . Then, we define precision and recall based on this length:

$$\text{Prec} = \frac{\text{LCS}(\hat{y}, y)}{|\hat{y}|} \quad (\text{num tokens in AI output}) \quad (17)$$

$$\text{Rec} = \frac{\text{LCS}(\hat{y}, y)}{|y|} \quad (\text{num tokens in reference}) \quad (18)$$

The ROUGE-L score is the F-score of these values (typically with $\beta = 1$):

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \text{Prec} \cdot \text{Rec}}{\text{Rec} + \beta^2 \text{Prec}} \quad (19)$$

Longest common subsequence score for summarization: ROUGE-L

Example (Step-by-Step Calculation of ROUGE-L)

Setting:

- Reference y = “the quick brown fox” (4 tokens)
- AI output $\hat{y}$ =“quick brown fox” (3 tokens)

Longest common subsequence score for summarization: ROUGE-L

Example (Step-by-Step Calculation of ROUGE-L)

Setting:

- Reference y = “the quick brown fox” (4 tokens)
- AI output $\hat{y}$ =“quick brown fox” (3 tokens)

LCS: The longest common subsequence is “quick brown fox”. $LCS = 3$.

Longest common subsequence score for summarization: ROUGE-L

Example (Step-by-Step Calculation of ROUGE-L)

Setting:

- Reference y = “the quick brown fox” (4 tokens)
- AI output $\hat{y}$ = “quick brown fox” (3 tokens)

LCS: The longest common subsequence is “quick brown fox”. $LCS = 3$.

Calculation ($\beta = 1$):

$$\text{Prec} = \frac{3}{3} = 1$$

$$\text{Rec} = \frac{3}{4} = 0.75$$

$$F_1 = \frac{2 \cdot 1 \cdot 0.75}{1 + 0.75} = \frac{1.5}{1.75} \approx 0.857.$$

Longest common subsequence score for summarization: ROUGE-L

Exercise (LCS Exercise)

Let reference $y = \text{"a b c d"}$ and AI output $\hat{y} = \text{"a c d"}$. Calculate ROUGE-L (F_1) step-by-step.

Longest common subsequence score for summarization: ROUGE-L

Answer

LCS:

- The longest common subsequence is “a c d”. $LCS = 3$

Longest common subsequence score for summarization: ROUGE-L

Answer

LCS:

- The longest common subsequence is “a c d”. $LCS = 3$

Lengths:

- AI output length $|\hat{y}| = 3$.
- Reference length $|y| = 4$.

Longest common subsequence score for summarization: ROUGE-L

Answer

LCS:

- The longest common subsequence is “a c d”. $LCS = 3$

Lengths:

- AI output length $|\hat{y}| = 3$.
- Reference length $|y| = 4$.

Metric Calculation ($\beta = 1$):

$$\text{Prec} = 3/3 = 1$$

$$\text{Rec} = 3/4 = 0.75$$

$$F_1 = \frac{2 \cdot 1 \cdot 0.75}{1 + 0.75} = \frac{1.5}{1.75} \approx 0.857.$$

What if we want to consider lexical semantic similarity? - BERTScore

Can we consider lexical semantic similarities that do not appear in sequences of letters directly?

What if we want to consider lexical semantic similarity? - BERTScore

Can we consider lexical semantic similarities that do not appear in sequences of letters directly?

As long as we rely on matching between tokens or characters, we cannot capture similarity between semantically similar but different words.

What if we want to consider lexical semantic similarity? - BERTScore

Can we consider lexical semantic similarities that do not appear in sequences of letters directly?

As long as we rely on matching between tokens or characters, we cannot capture similarity between semantically similar but different words.

Idea: Use **vector representations** in an established neural network, and replace matching with **vector similarity**.

What if we want to consider lexical semantic similarity? - BERTScore

Can we consider lexical semantic similarities that do not appear in sequences of letters directly?

As long as we rely on matching between tokens or characters, we cannot capture similarity between semantically similar but different words.

Idea: Use **vector representations** in an established neural network, and replace matching with **vector similarity**.

This idea leads us to the definition of **BERTScore** [11].

What if we want to consider lexical semantic similarity? - BERTScore

If you naively report the average similarity, the behavior of frequent but semantically unimportant words (e.g., “of,” “the,” etc.) can become too dominant.

What if we want to consider lexical semantic similarity? - BERTScore

If you naively report the average similarity, the behavior of frequent but semantically unimportant words (e.g., “of,” “the,” etc.) can become too dominant.

IDF weighting emphasizes information-rich words and relatively reduces the contribution of **common words**.

What if we want to consider lexical semantic similarity? - BERTScore

If you naively report the average similarity, the behavior of frequent but semantically unimportant words (e.g., “of,” “the,” etc.) can become too dominant.

IDF weighting emphasizes information-rich words and relatively reduces the contribution of **common words**.

Definition (Inverse Document Frequency (IDF))

Let $\mathcal{D}$ be a collection of documents. The **inverse document frequency** for a token u is:

$$\text{idf}_{\mathcal{D}}(u) := \log\left(\frac{|\mathcal{D}| + 1}{\text{df}_{\mathcal{D}}(u) + 1}\right)$$

where $\text{df}_{\mathcal{D}}(u)$ is the number of documents in $\mathcal{D}$ containing token u .

What if we want to consider lexical semantic similarity? - BERTScore

Definition (Complete Rigorous Definition of BERTScore (F_1))

Let the token representations for the AI output be $\{\tilde{h}_i\}$ and for the reference be $\{\tilde{g}_j\}$ (normalized to unit length).

What if we want to consider lexical semantic similarity? - BERTScore

Definition (Complete Rigorous Definition of BERTScore (F_1))

Let the token representations for the AI output be $\{\tilde{h}_i\}$ and for the reference be $\{\tilde{g}_j\}$ (normalized to unit length). The similarity matrix is $s_{i,j} := \tilde{h}_i^\top \tilde{g}_j$.

What if we want to consider lexical semantic similarity? - BERTScore

Definition (Complete Rigorous Definition of BERTScore (F_1))

Let the token representations for the AI output be $\{\tilde{h}_i\}$ and for the reference be $\{\tilde{g}_j\}$ (normalized to unit length). The similarity matrix is $s_{i,j} := \tilde{h}_i^\top \tilde{g}_j$. The precision and recall are calculated as IDF-weighted sums of maximal similarities:

$$\text{Prec}_{\text{BERT}} = \frac{\sum_i \text{idf}(\hat{w}_i) \cdot \max_j s_{i,j}}{\sum_i \text{idf}(\hat{w}_i)} \quad (20)$$

$$\text{Rec}_{\text{BERT}} = \frac{\sum_j \text{idf}(w_j) \cdot \max_i s_{i,j}}{\sum_j \text{idf}(w_j)} \quad (21)$$

What if we want to consider lexical semantic similarity? - BERTScore

Definition (Complete Rigorous Definition of BERTScore (F_1))

Let the token representations for the AI output be $\{\tilde{\mathbf{h}}_i\}$ and for the reference be $\{\tilde{\mathbf{g}}_j\}$ (normalized to unit length). The similarity matrix is $s_{i,j} := \tilde{\mathbf{h}}_i^\top \tilde{\mathbf{g}}_j$. The precision and recall are calculated as IDF-weighted sums of maximal similarities:

$$\text{Prec}_{\text{BERT}} = \frac{\sum_i \text{idf}(\hat{w}_i) \cdot \max_j s_{i,j}}{\sum_i \text{idf}(\hat{w}_i)} \quad (20)$$

$$\text{Rec}_{\text{BERT}} = \frac{\sum_j \text{idf}(w_j) \cdot \max_i s_{i,j}}{\sum_j \text{idf}(w_j)} \quad (21)$$

F_1 Aggregation:
$$F1_{\text{BERT}} = \frac{2 \text{Prec}_{\text{BERT}} \cdot \text{Rec}_{\text{BERT}}}{\text{Prec}_{\text{BERT}} + \text{Rec}_{\text{BERT}}}.$$

Note: we can use any vector in the NN related to the token.

What if we want to consider lexical semantic similarity? - BERTScore

Example (Complete Manual Calculation of BERTScore (F_1))

Setting:

- Reference y = “red apple”
- AI output $\hat{y}$ = “the red apples”
- IDF is disabled for simplicity ($u_i = v_j \equiv 1$).
- Word embeddings (already ℓ_2 -normalized) are given as:

the = (0, 0, 1)

red = (1, 0, 0)

apple = (0, 1, 0)

apples = (0, 0.8, 0.6)

What if we want to consider lexical semantic similarity? - BERTScore

Step 1: Similarity Matrix $s_{i,j}$ (rows: AI output, columns: reference):

The similarity is the dot product of the normalized vectors.

	red (1, 0, 0)	apple (0, 1, 0)
the (0, 0, 1)	0	0
red (1, 0, 0)	1	0
apples (0, 0.8, 0.6)	0	0.8

What if we want to consider lexical semantic similarity? - BERTScore

Step 2: Precision

For each AI output token (row), find its maximum similarity with any reference token (column). Then average these maximums.

	red	apple	$\max_j s_{i,j}$
the	0	0	0
red	1	0	1
apples	0	0.8	0.8

$$\text{Prec}_{\text{BERT}} = \frac{0 + 1 + 0.8}{3} = 0.6.$$

What if we want to consider lexical semantic similarity? - BERTScore

Step 3: Recall

For each reference token (column), find its maximum similarity with any AI output token (row). Then average these maximums.

	red	apple
the	0	0
red	1	0
apples	0	0.8
$\max_i s_{i,j}$	1	0.8

$$\text{Rec}_{\text{BERT}} = \frac{1 + 0.8}{2} = 0.9.$$

What if we want to consider lexical semantic similarity? - BERTScore

Step 4: F_1 Score

Now we calculate the harmonic mean of Precision and Recall.

$$F1_{\text{BERT}} = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}} = \frac{2 \cdot 0.6 \cdot 0.9}{0.6 + 0.9} = \frac{1.08}{1.5} = 0.72.$$

What if we want to consider lexical semantic similarity? - BERTScore

Exercise (BERTScore (F_1) Exercise)

Given word embeddings (normalized) as:

$$\text{fast} = (1, 0, 0)$$

$$\text{car} = (0, 1, 0)$$

$$\text{quick} = (0.9, 0.1, 0)$$

$$\text{automobile} = (0, 1/\sqrt{2}, 1/\sqrt{2})$$

Let reference y = “fast car” and AI output $\hat{y}$ = “quick automobile”. IDF is disabled. Calculate BERTScore (F_1) step-by-step.

What if we want to consider lexical semantic similarity? - BERTScore

Answer

Similarity Matrix $s_{i,j}$	fast	car
quick	0.9	0.1
automobile	0	$1/\sqrt{2} \approx 0.707$

What if we want to consider lexical semantic similarity? - BERTScore

Answer

Similarity Matrix $s_{i,j}$	fast	car
quick	0.9	0.1
automobile	0	$1/\sqrt{2} \approx 0.707$

Precision (avg of row maxes):

$$\text{Prec} = \frac{\max(0.9, 0.1) + \max(0, 1/\sqrt{2})}{2} = \frac{0.9 + 1/\sqrt{2}}{2} \approx 0.8036.$$

What if we want to consider lexical semantic similarity? - BERTScore

Answer

Similarity Matrix $s_{i,j}$	fast	car
quick	0.9	0.1
automobile	0	$1/\sqrt{2} \approx 0.707$

Precision (avg of row maxes):

$$\text{Prec} = \frac{\max(0.9, 0.1) + \max(0, 1/\sqrt{2})}{2} = \frac{0.9 + 1/\sqrt{2}}{2} \approx 0.8036.$$

Recall (avg of col maxes):

$$\text{Rec} = \frac{\max(0.9, 0) + \max(0.1, 1/\sqrt{2})}{2} = \frac{0.9 + 1/\sqrt{2}}{2} \approx 0.8036.$$

What if we want to consider lexical semantic similarity? - BERTScore

Answer

Similarity Matrix $s_{i,j}$	fast	car
quick	0.9	0.1
automobile	0	$1/\sqrt{2} \approx 0.707$

Precision (avg of row maxes):

$$\text{Prec} = \frac{\max(0.9, 0.1) + \max(0, 1/\sqrt{2})}{2} = \frac{0.9 + 1/\sqrt{2}}{2} \approx 0.8036.$$

Recall (avg of col maxes):

$$\text{Rec} = \frac{\max(0.9, 0) + \max(0.1, 1/\sqrt{2})}{2} = \frac{0.9 + 1/\sqrt{2}}{2} \approx 0.8036.$$

F₁: Since $\text{Prec} = \text{Rec}$, $F1_{\text{BERT}} = \text{Prec} = \text{Rec} \approx 0.8036$.

Evaluating natural language outputs - answer

Question: How can we directly evaluate natural language outputs with semantically similar answers considered?

Answer: We standardize the answer format (MCQ, numerical answer), subsequence-based similarity (BLEU, chrF, longest common sequence), NN weight representation similarity (BERTScore) etc.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
5. How do we evaluate a probabilistic language model?
- 6. How do we evaluate natural language outputs?**
7. Summary
8. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
5. How do we evaluate a probabilistic language model?
6. How do we evaluate natural language outputs?
- 7. Summary**
8. Next Time

Summary

Let's summarize the key points corresponding to the learning objectives of this lecture.

- **Non-triviality:** Natural language output has infinitely many semantically equivalent solutions, making the application of classical evaluation methods difficult.

Summary

Let's summarize the key points corresponding to the learning objectives of this lecture.

- **Non-triviality**: Natural language output has infinitely many semantically equivalent solutions, making the application of classical evaluation methods difficult.
- **Distinction**: We clearly distinguished between the **evaluation of the language model itself** (PPL, most likely option) and the **evaluation of the string output** (EM/F1, BLEU, ROUGE-L, chrF, BERTScore, Numeric Accuracy).

Summary

Let's summarize the key points corresponding to the learning objectives of this lecture.

- **Non-triviality:** Natural language output has infinitely many semantically equivalent solutions, making the application of classical evaluation methods difficult.
- **Distinction:** We clearly distinguished between the **evaluation of the language model itself** (PPL, most likely option) and the **evaluation of the string output** (EM/F1, BLEU, ROUGE-L, chrF, BERTScore, Numeric Accuracy).
- **Application:** We rigorously defined each metric and understood the calculation procedures through concrete examples and exercises.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
5. How do we evaluate a probabilistic language model?
6. How do we evaluate natural language outputs?
- 7. Summary**
8. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. The Importance of Automatic Evaluation
4. What do we evaluate?
5. How do we evaluate a probabilistic language model?
6. How do we evaluate natural language outputs?
7. Summary

8. Next Time

Next time

In this lecture, we dealt with **absolute** evaluation metrics that target a single probabilistic language model.

Next time

In this lecture, we dealt with **absolute** evaluation metrics that target a single probabilistic language model.

In the next lecture, we will address **relative** evaluation metrics that quantify the **deviation** from a given **reference probabilistic language model** (e.g., distances and divergences between distributions).

References i

- [1] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord.
Think you have solved question answering? try arc, the ai2 reasoning challenge.
In Proc. of NAACL-HLT, 2018.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman.
Training verifiers to solve math word problems.
In arXiv preprint arXiv:2110.14168, 2021.
GSM8K dataset.

References ii

- [3] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt.

Measuring massive multitask language understanding.

In International Conference on Learning Representations (ICLR), 2021.

- [4] Chin-Yew Lin.

Rouge: A package for automatic evaluation of summaries.

In Proc. of Workshop on Text Summarization Branches Out, 2004.

- [5] Stephanie Lin, Jacob Hilton, and Owain Evans.

Truthfulqa: Measuring how models mimic human falsehoods.

In Proc. of ACL, 2022.

References iii

- [6] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher.
Pointer sentinel mixture models.
In International Conference on Learning Representations (ICLR) Workshop, 2017.
Introduces WikiText datasets (WikiText-2/WikiText-103).
- [7] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu.
Bleu: a method for automatic evaluation of machine translation.
In Proceedings of the 40th annual meeting of the Association for Computational Linguistics, pages 311–318, 2002.

References iv

- [8] Maja Popović.
chrf: Character n-gram f-score for automatic mt evaluation.
In Proc. of WMT, 2015.
- [9] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang.
Squad: 100,000+ questions for machine comprehension of text.
In Proc. of EMNLP, 2016.
- [10] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi.
Hellaswag: Can a machine really finish your sentence?
In Proc. of ACL, 2019.

References v

[11] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi.

Bertscore: Evaluating text generation with bert.

In Proc. of ICLR, 2020.